

CSC240 Text Processing Project Preview Report

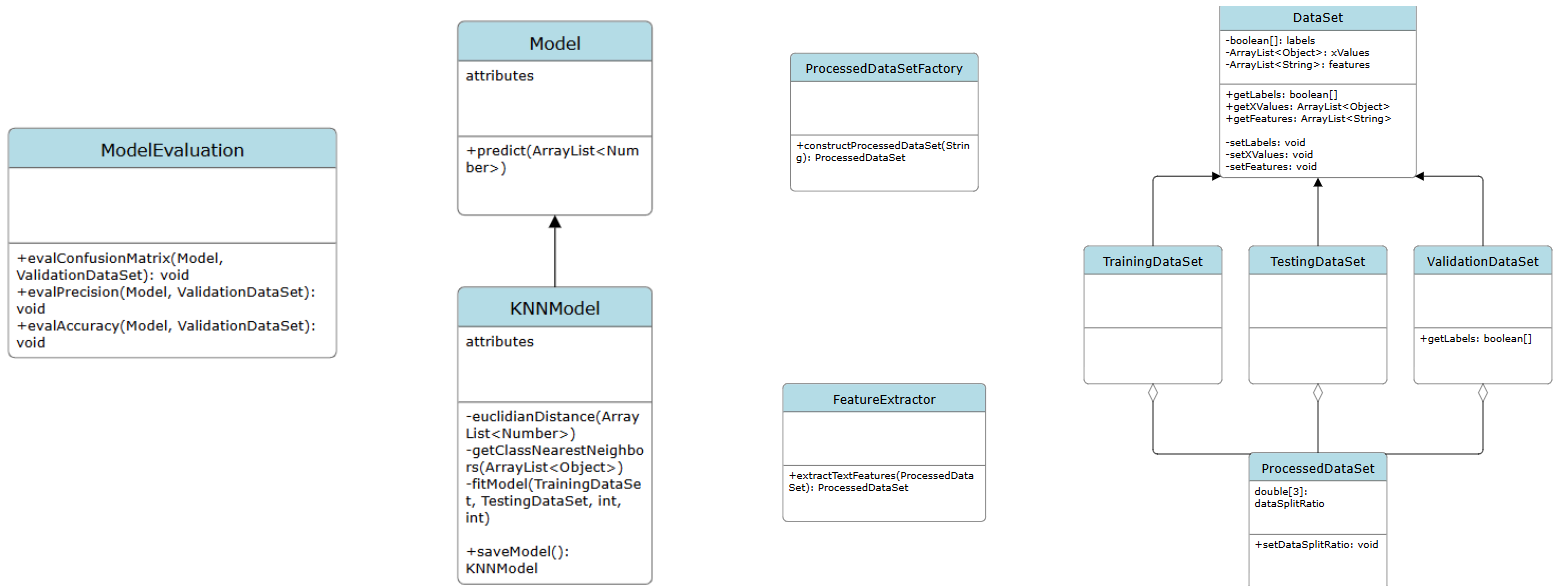
Due: 3.27.2025

Jae Elizabeth Giesen, Oleg

Approach:

- **KNN**/Naive Bayes
- Feature extraction
 - Manually. Check existing research to determine features to extract.
 - # of capitalized words
 - # of words containing numbers, non-Latin characters
 - Clever regex?
 - Occurrence of each word
 - Dimensionality reduction?
- Outlier classification
 - Discard unlabeled data

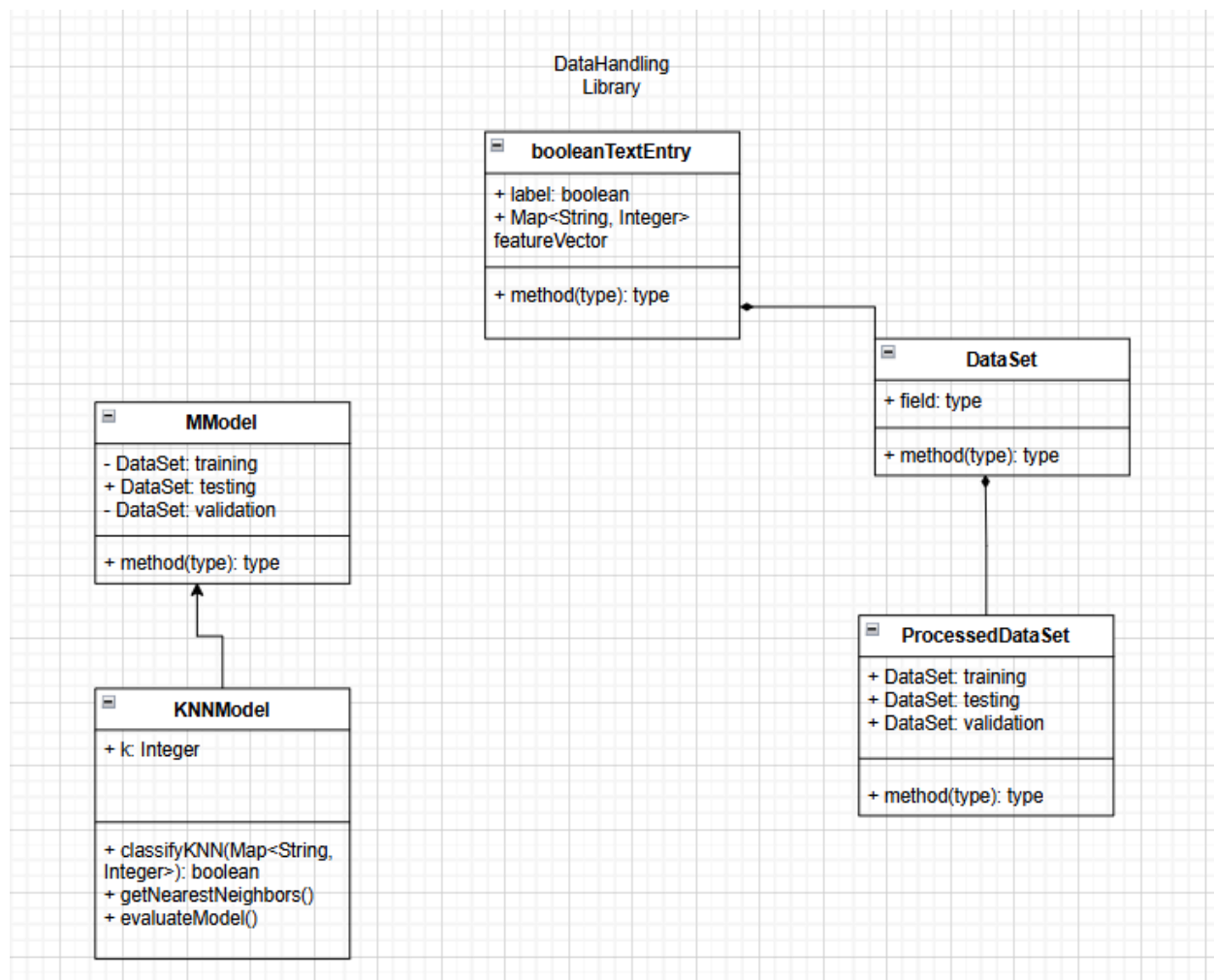
UML/Pseudocode:



1. Read email dataset into **ProcessedDataSet**. Split with ratio of 70%:15%:15%.
2. Extract the following features, convert to numeric data:
 - a. Occurrence of words:
 - i. Discard words with $\leq n$ occurrences?
 - b. Number of capitalized words
 - c. Number of words containing numeric characters
 - d. Transform and data?

3. Fit model to training dataset...
4. Compare unknown emails via their proximity in Euclidian (Manhattan is expensive + not used in KNN, dimensions can be directly compared) distance to training exemplars
 - a. Process features from email
 - b. Discard features with limited/no data?
 - c. Calculate similarity to neighbors
 - d. Let exemplars vote (weighted by proximity? Proximity cutoff?)
 - e. Highest vote wins? Heaviest vote wins?
5. Save fitted model out...
6. Validate with ValidationDataSet and Evaluator

ALGORITHM FINAL IMPLEMENTATION NOTES



KNN Implementation

1. Load data, split into training, testing, validation sets. Read in emails line-by-line, discard header. Emails are split into tokens, which are counted into a `Map<String, Integer>`,

representing the email as a feature vector. The label is read, and the feature vector and label are stored in an object called a `booleanTextObject`.

2. `DataSet` comprises `ArrayLists` of `booleanTextObjects`, with accessor and mutator methods.
3. To classify an email, we compare a given email, transformed into a feature vector, against all emails in the training set to get an array of Euclidean distance values.
4. Sort the array and retrieve the *k* closest neighbors (first *k* entries)
5. *K* neighbors vote; if more true than false; eval to true. If more false than true; eval to false. If equal; nearest neighbor wins.

Post-discussion

Implementing the original UML proved to be an arduous task that led to lots of interlinked and unwieldy code, so it was simplified and restarted twice. The final UML, containing the classes `MModel`, `KNNModel`, `DataSet`, `booleanTextObject`, and `ProcessedDataSet` drastically simplified the implementation. `MModel` could be eliminated for the sake of reducing complexity, but should I wish to extend it, say for a Naive Bayes classifier, classification evaluation logic could be implemented as abstract functions in `MModel`. Simplifying `DataSet` was an unfortunate decision made in the name of reducing complexity and complying with deadlines, as I would have liked to have genericized/template classes. While the `booleanTextObject` works well as an abstraction for text objects with 2 states, it should be genericized to accept an enum instead of boolean. However, I learned how to reduce space complexity drastically by changing my bag of words approach to use a `Map` instead of `ArrayList` for each `booleanTextObject`. This is somewhat computationally inefficient with my implementation of the `.getKNearestNeighbors()` method, but it works, is cleanly readable, and runs on limited memory space. In the future, I would like to have a better understanding of handling generics such that different classifiers can be cleanly implemented, and `DataSet` can be extended to handle numeric entries.